Ad Soyad: _____	**Tarih:** _____
Bölüm 11 – Ünite 3 – *İlerleme Takibi ve İptal – Çalışma Kağıdı*	

Süre

Bu çalışma kağıdını tamamlamak için tahmini süre: **45–50 dakika**.

Bölüm 1: Boşluk Doldurma

(5 dakika – 10 puan)

1. İlerleme sayacı için std:: _____ kullanmak std::mutex'ten daha hafiftir; sayma için memory_order _____ yeterlidir.
2. C++20 işbirlikçi iptal mekanizmasının üç ana bileşeni: std::stop_ _____, std::stop_ _____, std::stop_ _____.
3. QProgressDialog::setMinimumDuration(_____) ile kısa süreli işlerde dialog'un açılması engellenir.
4. Worker'dan UI'ya sinyal gönderirken Qt otomatik olarak `Qt::` _____ bağlantı türünü kullanır; iptal sinyali için ise `Qt::` _____ tercih edilir.
5. Bekleyen std::condition_variable'ı iptal anında uyandırmak için std::stop_ _____ kullanılır.

Bölüm 2: Çoktan Seçmeli

(5 dakika – 10 puan)

1. İlerleme sinyali ne sıklıkla emit edilmeli?
   - A) Her satırda
   - B) Her 50-100 satırda bir
   - C) İş bittiğinde tek seferde
   - D) Hiç emit edilmez
2. C++20 jthread'in thread'e karşı en önemli avantajı?
   - A) Daha hızlı bağlam değiştirme
   - B) Otomatik join + iptal desteği + istisna güvenliği
   - C) Daha az bellek kullanımı
   - D) Yalnızca Linux'ta çalışır
3. Memory order seçimi: ilerleme sayacında ve "bitti" bayrağında sırasıyla?
   - A) seq_cst, seq_cst (en güvenli)
   - B) relaxed, release/acquire
   - C) acquire, release
   - D) Hiç memory_order belirtmemek
4. İptal kontrolü ne sıklıkla yapılmalı?
   - A) Her satırda (CPU israfı)
   - B) Her 10000 satırda (kullanıcı 5+ saniye bekler)

C) Yaklaşık 100 ms aralıklarla
D) Hiç gerekli değil

5. QProgressDialog ile worker arasındaki iptal bağlantısı neden `Qt::DirectConnection` olmalı?

   A) Daha hızlı çalışsın
   B) request_stop thread-safe; iptalin worker'a anında ulaşması için
   C) Qt'nin tek desteklediği bağlantı türü olduğu için
   D) Hata olmasın diye

Bölüm 3: Doğru/Yanlış

(5 dakika – 10 puan)

1. Atomic sayaç mutex'ten daha hızlıdır çünkü thread'i uyutmaz, bağlam değiştirmez.
2. std::stop_callback, iptal istendiği anda kayıtlı lambda'yı çalıştırır.
3. Cache line çekişmesi (false sharing), atomic sayaca çok yüksek hızda yazarken görünebilen bir problemdir.
4. QProgressDialog::wasCanceled metodu, kullanıcının iptal'e basıp basmadığını söyler.
5. jthread'in destructor'ı önce join çağırır, sonra request_stop.

Bölüm 4: Kod Analizi

(15 dakika – 30 puan)

1. Aşağıdaki kod neden iptal'e tepki vermez?

```
std::jthread t([] {
    while (true) {
        std::this_thread::sleep_for(std::chrono::milliseconds(50));
        std::cout << "tik\n";
    }
});

std::this_thread::sleep_for(std::chrono::milliseconds(200));
t.request_stop();
```

3

2. Aşağıdaki Qt sinyal/slot kodunda kullanıcı iptal'e basınca ne olur?

```
connect(dialog, &QProgressDialog::canceled, worker,
        &Worker::durdur, Qt::QueuedConnection);  // Queued

// Worker'in calistir() metodu uzun bir for dongusu icinde,
// hicbir zaman event loop calismaz.
```

3

Bölüm 5: Kod Yazma

(15 dakika – 40 puan)

1. std::atomic ile bir worker thread oluşturun: 0'dan 1000'e kadar sayar, ana thread her 100 ms'de sayacı okuyup ekrana yüzde yazar. Worker bittiğinde ana thread döngüden çıkmalı. 8cm